

JobAgent

Smart Career Assistant

Complete Technical Guide: The Languages and ML Models Behind Every Function in the Site

Generated 10 Sep 2026 | Prepared from the project source code (app.py, static/js, utils, models, scrapers, n8n workflows)

Table of Contents

- 1. Project Overview & Architecture
- 2. Technology & Programming Languages
- 3. ML & AI Models Used
- 4. Frontend Functions (JavaScript / HTML / CSS)
- 5. Backend Functions (Python / Flask)
- 6. Core AI/ML Analysis Engines in Detail
- 7. Authentication & Security Functions
- 8. Job Scraping Subsystem
- 9. Community Chat & Dashboard Functions
- 10. Database Layer (SQLAlchemy Models)
- 11. n8n AI Orchestration Workflows
- 12. Hardware-Constrained Architecture Plan & Proposed Local AI Stack
- 13. Appendix - Sibling Projects in the Workspace

1. Project Overview & Architecture

JobAgent is a full-stack web application that works as an AI-assisted career assistant. A user can search any job role and instantly receive a detailed career guide, find related job titles, analyze their resume with an ATS (Applicant Tracking System) scored report, discover skill gaps with a learning roadmap, practice interview questions, view market career insights, save everything to a personal dashboard, and join a moderated community chat.

The project also contains a command-line job scraping agent (main.py + scrapers/) that pulls live job listings from Naukri, RemoteOK and Wellfound into a CSV file, plus five n8n automation workflows that run the same role guides through an AI Large Language Model (gemini-pro) via webhooks.

High-level layered architecture:

- **Client layer:** HTML5 + CSS3 SPAs served by Jinja2 templates (index.html, login.html, etc.), JavaScript (vanilla, no framework) via static/js/app.js and static/js/auth.js, Font Awesome icons and the Inter web font.
- **Server layer:** Python 3.9 + Flask 3.0 (REST JSON APIs under /api/*), Flask-Login sessions, Flask-SQLAlchemy.
- **AI/ML layer:** Google Gemini (gemini-pro) as the primary LLM; Hugging Face inference (facebook/bart-large-mnli zero-shot classifier) as a fallback; an internal, rule-based ATS scoring engine; regex/NLP-lite parsing utilities.
- **Data layer:** SQLite by default (jobagent.db), PostgreSQL-ready via DATABASE_URL, 13+ mapped models.
- **Automation layer:** n8n workflows that expose AI webhooks for resume analysis, interview prep, career insights, job recommendations and feedback processing.

2. Technology & Programming Languages

These are the programming languages and core technologies used across the whole site, together with the exact job they perform inside JobAgent.

Language / Tech	Version	Where It Lives	What It Does Here
Python	3.9+	app.py, main.py, utils/, models/, scrapers/	All backend logic: web server, REST APIs, ATS scoring engine, and more.
JavaScript (vanilla ES6+)	-	static/js/app.js, auth.js	All client-side interactivity: SPA page switching, fetch() API calls, and more.
HTML5	-	templates/*.html	Page structure for the SPA (index.html) and auth pages (login, sign up, etc.).
CSS3	-	static/css/style.css, static/css/auth.css	Premium dark SaaS theme, responsive layout, grids, cards, progress bars, and more.
Flask (Python micro-framework)	3.0.0	app.py	Web framework routing every URL; JSON API responses, session management, and more.
SQL / SQLAlchemy ORM	2.0.23	models/*.py	Relational persistence for users, jobs, chat, audit logs, tokens, dashboard data, and more.
Jinja2 (templating)	bundled with Flask	templates/	Server-side template rendering for the HTML pages.
Bash / Batch	-	start_server.bat (job-platform)	Startup scripts that launch the servers on Windows.
JSON	-	config files, n8n/workflows/*.json	Configuration and the n8n AI workflow definitions.

Key Python libraries in requirements.txt: pytest (tests), beautifulsoup4 + lxml + Playwright + Firecrawl (scraping), pandas + numpy (data), requests + httpx + aiohttp (HTTP APIs), google-generativeai + openai + huggingface-hub (AI/ML), python-docx + PyPDF2 + pdfplumber (resume documents), passlib[bcrypt] + python-jose + email-validator (security), tenacity (retries) and cachetools (caching).

3. ML & AI Models Used

JobAgent combines three kinds of intelligence. The first is a commercial LLM accessed over the network (Google Gemini). The second is a fine-tuned open-source transformer model served by the Hugging Face inference API. The third is an in-house, transparent "expert-system" engine written in pure Python (weighted scoring + regular expressions) that mimics the behaviour of a trained ML model and works fully offline.

Model / AI	Provider / Runtime	Tasks It Powers	Type
gemini-pro	Google AI (generativelanguage API)	Resume analysis, interview questions generation, career insights, LLM (decoder-only)	
facebook/bart-large-mnli	Hugging Face Inference API hd-infer	Zero-shot classification of resume sections (skills, experience, education)	Transformer, encoder-decoder, zero-shot NLP
GPT-4 (optional)	OpenAI API (openai 1.6.1 client)	Advanced reasoning / complex analysis enhancement	Transformer LLM
ANS/ATS Scoring Engine	In-process Python, app.py analyze_weighted_text()	Multi-factor resume score, readability score	Rule-based expert system (heuristic ML)
Skill Extractor	In-process Python, _extract_skills_from_text()	Detects 1050+ known skills from free text	Dictionary / regex pattern matcher
Query Parser	Python, utils/query_parser.py	Turns natural language like "product manager role" into structured JSON	Regular expression ML-style parser

3.1 Google Gemini (gemini-pro)

This is the primary Large Language Model. The code in `utils/api_integrations.py` configures the official SDK with `genai.configure(api_key=...)` and instantiates `genai.GenerativeModel('gemini-pro')`. For resume analysis it is prompted to output a detailed JSON report (ATS score, keyword score, strengths, weaknesses, missing keywords and suggestions). For interviews it generates 5 realistic questions per role with answers, difficulty and timing. The same model is reused inside the `n8n` resume-analysis and interview-prep workflows through the Google AI node, so the LLM capability works both from Python and from the automation layer.

3.2 Hugging Face - bart-large-mnli

Used as a zero-shot text classifier with multi-label support. The resume text is sent to `https://api-inference.huggingface.co/models/facebook/bart-large-mnli` with candidate labels such as "has technical skills", "has work experience", "has education", "needs improvement". This provides an independent ML opinion about the presence of each resume section, acting as a fallback path when the Gemini key is not configured.

3.3 In-process Heuristic Engines

The most heavily used "ML" in the live site is the rule-based analysis engine. `analyze_resume_text()` computes a 100-point ATS score from ten weighted components (section detection 10%, keywords 30%, experience 20%, contact 5%, skills 15%, education 5%, projects 10%, grammar 10% scaled, length 5%, ATS compatibility 5%). Every component is a transparent function of regular-expression features, so the same input always produces the same score - ideal for a scoring product. The skill extractor and the natural-language query parser follow the same philosophy with heuristics instead of neural weights.

4. Frontend Functions (JavaScript / HTML / CSS)

All frontend logic lives in `static/js/app.js` (the career assistant) and `static/js/auth.js` (login/signup). The UI is a single HTML page (`index.html`) divided into sections with class toggling - no frontend framework is used. Every function talks to the Flask API through the `fetch()` wrapper below.

4.1 Core UI plumbing

showPage(pageName)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript (vanilla DOM API)
ML / AI Model	None - plain DOM manipulation

Implements client-side single-page-app navigation. It removes the active class from all `.page` sections and `.nav-link` items, adds it to the requested section, closes the mobile menu, scrolls to the top, and lazily loads dashboard or chat content when those pages are opened.

toggleNav()

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript
ML / AI Model	None - CSS class toggle

Opens and closes the responsive hamburger menu on small screens by toggling the `open` class on the navigation links container.

showToast(message, type)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript
ML / AI Model	None - pure DOM/CSS animation

Creates a temporary toast notification element, appends it to the body, auto-removes it after 3 seconds. Supports success and error variants using Font Awesome icons plus CSS animation.

apiCall(url, options)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript (async/await + fetch)
ML / AI Model	None - HTTP client wrapper

A promise-based wrapper around `fetch()` used by every other frontend function. It sends JSON headers, throws a descriptive error when the response is not OK, and parses JSON responses. This single function is the gateway between the browser and all the Python/Flask endpoints.

4.2 Search / Role-guide workflow

heroSearch() and quickSearch(role)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript
ML / AI Model	None - input plumbing

Captures the text typed in the hero search box, copies it into the main search input, navigates to the search page and triggers `performSearch()`. `quickSearch` fills the hero box first so the popular-role chips work with one click.

performSearch()

Route / File	<code>POST /api/expand-roles</code> and <code>POST /api/search</code>
Language / Tech	JavaScript + calls Python APIs
ML / AI Model	None directly - the Python side uses dictionary matching and AI-assisted role guides

The main search workflow. It shows a loading spinner, asks the backend to expand the query into related role titles (`/api/expand-roles`), renders those as clickable chips, then asks for the full role guide pack (`/api/search`) which contains the primary guide, related guides (top 5) and the platform search URLs.

showRoleExpansion(roles) and exploreRole(role)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript
ML / AI Model	None - display + re-trigger search

Renders the expanded role list as tag chips with the total role count. Clicking a chip calls `exploreRole`, which replaces the search input with that role and runs `performSearch` again, giving a drill-down experience without a full page reload.

renderGuides(data) and renderPlatformButtons(urls, query)

Route / File	<code>static/js/app.js</code>
Language / Tech	JavaScript
ML / AI Model	None - templating/rendering logic

Renders the results header (role title and subtitle), the guide cards grid showing overview, responsibilities, required and preferred skills, salary range, interview topics, key industries, and buttons that deep-link directly to LinkedIn, Indeed, Glassdoor and 14 other job boards with the role pre-filled.

4.3 Resume analysis UI

analyzeResume()	
Route / File	POST /api/analyze-resume
Language / Tech	JavaScript + multipart/form-data POST
ML / AI Model	Uses the backend ATS engine (heuristic) plus optional file parsing; AI enhancement available via Gemini path

Builds a FormData object with the pasted resume text, optional target role and an optional uploaded .txt/.pdf/.docx file, then sends it to the Flask endpoint. The response contains the full ATS report which the UI then renders with scores, strengths, weaknesses and suggestions.

renderResumeAnalysis(data)	
Route / File	static/js/app.js
Language / Tech	JavaScript
ML / AI Model	Display of the heuristic ATS scoring engine output; skills are detected by the regex skill extractor

Renders the ATS score badge, the component breakdown bars, detected sections, matched and missing keywords, action verbs found, readability level, strengths, weaknesses and improvement suggestions with priority and expected impact.

4.4 Interview preparation UI

loadInterviewQuestions()	
Route / File	POST /api/interview-prep
Language / Tech	JavaScript
ML / AI Model	Role-specific question bank from the Python dictionary INTERVIEW_QUESTIONS_DB; AI generation available through Gemini

Reads the selected role and fetches the question pack. The backend returns grouped questions (technical, behavioral, coding) with answer tips for each.

renderInterviewQuestions(data)	
Route / File	static/js/app.js
Language / Tech	JavaScript
ML / AI Model	None - rendering only

Groups questions by category with icons and counts, and renders expandable question cards that reveal answer tips when clicked.

4.5 Career insights UI

loadCareerInsights() and renderCareerInsights(data)

Route / File	POST /api/career-insights
Language / Tech	JavaScript
ML / AI Model	Data comes from the curated CAREER_INSIGHTS_DB dictionary; salary enhancement can use live APIs via utils/api_integrations.py

Requests salary, demand, growth and outlook data for a role and renders a grid of insight cards: average USD salary, INR salary range, demand level, growth rate, hiring trend, future outlook, top companies, key industries and entry/senior salary levels.

4.6 Dashboard UI

loadDashboard() and renderDashboard(data)

Route / File	GET /api/dashboard
Language / Tech	JavaScript
ML / AI Model	None - statistics aggregation by Flask from in-memory session store

Builds four stat cards (saved guides, resumes analyzed, skills tracked, searches) and lists saved role guides, ATS score history with colour-coded badges, recent search history and tracked skill progress bars.

4.7 Auth UI (auth.js)

handleSignup / handleLogin

Route / File	POST /api/auth/signup and /api/auth/login
Language / Tech	JavaScript
ML / AI Model	None - client-side validation only; security handled in Python (bcrypt hashing, email-validator)

Runs client-side validation (matching passwords, password strength meter, email format), shows inline field errors, disables the submit button while waiting and posts the credentials as JSON. On success it stores the user object and redirects to the dashboard.

handleForgotPassword / handleResetPassword

Route / File	POST /api/auth/forgot-password and /api/auth/reset-password
Language / Tech	JavaScript
ML / AI Model	None - token flow implemented in Python

Handles the two-step password recovery UX: request a reset link by email, then submit the new password together with the reset token from the URL. Shows success/error toasts in both steps.

handleLogout()

Route / File	POST /api/auth/logout
Language / Tech	JavaScript
ML / AI Model	None - session invalidation in Python

Calls the logout endpoint, clears the local user state and redirects to the home page.

5. Backend Functions (Python / Flask)

Every backend function below is a Python route handler in app.py. They receive JSON or file data, execute the business logic, and return JSON to the JavaScript frontend. Type hints (Dict, List, Any, Optional) are used throughout, and seven of them are decorated with @login_required so they only work for authenticated users.

5.1 Page routes (server-rendered HTML)

login_page(), signup_page(), forgot_password_page(), reset_password_page(token)

Route / File	GET /login, /signup, /forgot-password, /reset-password/<token>
Language / Tech	Python - Flask render_template
ML / AI Model	None - template rendering

Renders the four authentication HTML pages using the Jinja2 templating engine. The reset page receives the token from the URL so auth.js can submit it with the new password.

index() and spa_fallback(path)

Route / File	GET / and /<path:path>
Language / Tech	Python - Flask
ML / AI Model	None - SPA routing

Serves index.html for the root and for every client-side section URL (e.g. /chat, /dashboard, /resume), enabling browser deep links to SPA pages. Asset URLs under /static are exempt from the fallback.

serve_static(filename)

Route / File	GET /static/<path:filename>
Language / Tech	Python - Flask send_from_directory
ML / AI Model	None - static assets

Serves CSS, JavaScript, images and fonts from the static folder with proper MIME types, used by the HTML pages above.

5.2 Search and role-domain APIs

api_search()

Route / File	POST /api/search
Language / Tech	Python; dictionary lookups; string normalisation
ML / AI Model	No neural model in the default path - the AI-assisted role guide data lives in ROLE_GUIDES_DB ; the same capability is replicated by the n8n Gemini workflow

The most important endpoint. It resolves the query into related roles using `get_related_roles`, generates the primary role guide plus the top 5 related guides via `get_role_guide`, records the search in the user session history and returns the complete package including deep links to 14 job platforms.

api_expand_roles()

Route / File	POST /api/expand-roles
Language / Tech	Python
ML / AI Model	ROLE_MAP semantic grouping (expert-system over a curated dictionary of 10 role families)

Accepts a single role and returns all related job titles by looking up the **ROLE_MAP** dictionary (e.g. python developer expands to 18 titles). If the role is unknown, `_expand_generic` constructs sensible variants automatically.

5.3 Analysis and career APIs

api_analyze_resume()

Route / File	POST /api/analyze-resume
Language / Tech	Python; multipart file handling; PDF/DOCX decoding fallbacks
ML / AI Model	Invokes the heuristic ATS scoring engine (<code>analyze_resume_text</code>) - a 10-category weighted expert model; Gemini and Hugging Face versions exist as AI upgrade paths

Accepts resume text or an uploaded .txt/.pdf/.docx file (up to 16 MB), extracts plain text, runs the weighted ATS analysis, logs the score and resume version to the session dashboard, and returns the full JSON report: `ats_score`, `ranking`, `breakdown`, `sections`, `contact info`, `skills`, `keywords`, `readability`, `strengths`, `weaknesses`, `suggestions` and `skill gap`.

api_skill_gap()

Route / File	POST /api/skill-gap
Language / Tech	Python
ML / AI Model	Set-based skill matching plus a rule-based priority classifier that labels missing skills as critical/important/optional; learning roadmap drawn from LEARNING_RESOURCES knowledge base

Compares the user's skill list with the required skills for the target role, computes a match percentage, categorises missing skills by importance and returns a step-by-step learning roadmap (skill type, difficulty, estimated time and the best free learning resources).

api_interview_prep()

Route / File	POST /api/interview-prep
Language / Tech	Python
ML / AI Model	INTERVIEW_QUESTIONS_DB curated question bank (expert-written); on-demand AI generation with gemini-pro in utils/api_integrations.py

Returns the full interview question pack for the selected role grouped into technical, behavioral and coding categories, each question containing answer tips for the candidate.

api_career_insights()

Route / File	POST /api/career-insights
Language / Tech	Python
ML / AI Model	CAREER_INSIGHTS_DB curated market-data knowledge base; live salary data can be merged through FreeAPIIntegrations

Returns structured market intelligence for a role: average US salary, INR salary range, demand level, growth forecast, hiring trend, top hiring companies, key industries and entry/senior level salaries.

5.4 Dashboard session APIs

api_dashboard(), api_save_job(), api_remove_job(), and ATS history endpoints

Route / File	GET/POST /api/dashboard/*
Language / Tech	Python
ML / AI Model	None - in-memory session store keyed by UUID (get_session_id + get_user_data)

Persists the anonymous user's saved role guides, ATS score history, resume versions, tracked skill progress and search history inside the Flask session store, and exposes them to the dashboard page. Deduplication prevents the same job guide being saved twice.

5.5 Chat & moderation APIs

Chat policy, member, message and moderation endpoints

Route / File	/api/chat/policy, /api/chat/accept-policy, /api/chat/messages, /api/chat/members, /api/chat/post, moderation routes
Language / Tech	Python + SQLAlchemy (ChatMessage, ChatUserStatus models)
ML / AI Model	Rule-based moderation: message flagging keywords, ban/private status logic

A privacy-policy-gated group chat for logged-in users. First a policy screen must be accepted, then messages stream through a lightweight polling mechanism in app.js. Messages that violate the rules are flagged server-side, and users can be banned with an explanatory reason so the UI can block access. Members are listed with online status.

6. Core AI/ML Analysis Engines in Detail

These pure-Python functions are the analytical heart of the site. They are deterministic, explainable and require no GPU - a deliberate design choice that makes scores auditable.

analyze_resume_text(text, target_role) - the ATS engine

Route / File	app.py analyze_resume_text()
Language / Tech	Python - regex, statistics and weighted aggregation (no external ML library)
ML / AI Model	Heuristic expert system. Ten weighted components produce the 100-point ATS score, mimicking commercial tools like Jobscan and ResumeWorded (weights below).

Splits the resume into words and sentences; measures word count and average sentence length; detects 8 standard sections via regex; extracts email, phone, LinkedIn, GitHub and portfolio links; matches keywords against the targeted role; counts action verbs; checks grammar issues such as too many fragments or informal language (um/uh/basically); computes ATS compatibility; then combines: sections 10, contact 5, keywords 30, experience 20, skills 15, education 5, projects 10, grammar 10, length 5, ATS 5 (scaled). Produces a ranking band (Poor through Excellent) plus actionable suggestions with priority and estimated score impact.

_extract_skills_from_text() - skill NLP extractor

Route / File	app.py SKILLS_DATABASE
Language / Tech	Python - regular expressions generated dynamically from the SKILLS_DATABASE of 150+ skills in 10 categories
ML / AI Model	Dictionary/database matcher with word-boundary regex; a rule-based information-extraction model equivalent to a zero-shot tagger but fully offline

Lower-cases the text and scans for every known skill wrapped in word boundaries (so 'python' is not matched by 'pythons'). Returns a de-duplicated sorted skill list used by both the ATS engine and the skill-gap endpoint.

get_related_roles(), _expand_generic() - semantic role expansion

Route / File	app.py ROLE_MAP
Language / Tech	Python - dictionary index + text heuristics
ML / AI Model	Curated semantic graph: ROLE_MAP groups 200+ job titles into 10 role families by labour-market similarity

Looks up the canonical role family for any query, then returns all sibling job titles. For roles not in the map, _expand_generic manufactures a plausible list by combining the role stem with seniority and specialism keywords, so the search experience never returns empty.

QueryParser - natural-language job query parser

Route / File	<code>utils/query_parser.py</code>
Language / Tech	Python - regex/NLP-lite
ML / AI Model	Heuristic parser with a knowledge base of Indian cities and location/role indicator words (the NLP-lite cousin of a named entity recogniser)

Turns a sentence such as 'product manager role in bangalore' into a structured `ParsedQuery` with job title, location and keywords. It extracts the location first using indicator phrases and the city database, then strips role indicators and stop words to recover the job title, cleans extra spaces and returns the remaining meaningful words as keywords. It also builds Naukri, RemoteOK and Wellfound search URLs and queries from the parsed result.

FreeAPIIntegrations - AI extension layer

Route / File	<code>utils/api_integrations.py</code>
Language / Tech	Python - requests, google-generativeai, huggingface-hub
ML / AI Model	gemini-pro (primary LLM); facebook/bart-large-mnli (zero-shot fallback); Adzuna, JSearch, ESCO and O*NET data APIs

A single Python class that reads API keys from environment variables, adds a one-hour TTL cache with LRU helpers around every external call, exposes `analyze_resume_with_gemini` (with JSON extraction from the model output), `analyze_resume_with_huggingface` (multi-label zero-shot classification of resume sections), `generate_interview_questions_ai`, job search through Adzuna and JSearch, skills through ESCO, and reports which integrations are configured. Every method fails soft (returns empty results or an error dict) so the site keeps working without API keys.

7. Authentication & Security Functions

utils/auth.py implements enterprise-grade authentication on top of Flask-Login. The ML-relevant detail is that risk detection (brute-force lockout) is deterministic rule logic, while every password is hashed with bcrypt so no plaintext ever touches the database.

validate_password(password), validate_email_address(email)

Route / File	utils/auth.py
Language / Tech	Python - regex + email-validator library
ML / AI Model	None - rule-based policy checks (password entropy policy model)

Enforces a strict password policy: minimum 8 and maximum 128 characters, at least one uppercase, one lowercase, one digit and one special character, and rejection of common patterns (password, 123456, qwerty...). Email addresses are validated against RFC standards by the email-validator package.

sanitize_input(text), sanitize_json(data)

Route / File	utils/auth.py
Language / Tech	Python - regex
ML / AI Model	None - XSS defence rules

Strips HTML tags, removes script blocks and inline event handlers (onclick, onload, ...) to neutralise stored and reflected XSS before data is stored or rendered.

create_user_account(), authenticate_user()

Route / File	utils/auth.py (called by /api/auth/signup, /api/auth/login)
Language / Tech	Python - SQLAlchemy + werkzeug bcrypt
ML / AI Model	None - the auth risk model is rule-based: 5 failed attempts lock the account for 15 minutes (record_login_attempt)

Creates users with salted bcrypt hashes, a profile row and default preferences; logs every signup and login to the audit table with the IP address. authenticate_user applies the locked-until lockout, verifies the hash, and on success resets failed-attempt counters and records last_login.

initiate_password_reset(), reset_password()

Route / File	utils/auth.py (called by /api/auth/* password routes)
Language / Tech	Python - secrets.token_urlsafe + SQLAlchemy
ML / AI Model	None - cryptographic random tokens and expiry rules

Generates a single-use random token with a fixed expiry, stores it in the PasswordResetToken table and simulates sending it by email (logged to the console). reset_password validates the token lifecycle, applies password rules, re-hashes the new password, invalidates every active session of the user and writes an audit log entry.

get_user_dashboard_data(user)

Route / File	utils/auth.py (served by GET /api/auth/me)
Language / Tech	Python - SQLAlchemy aggregations
ML / AI Model	None - statistics queries

Collects saved jobs, applications and interviews for the logged-in user and computes derived statistics: total saved, total applications, total interviews, active applications and offers received.

8. Job Scraping Subsystem

The command-line agent (main.py + scrapers/) collects live job postings from three platforms and stores them in CSV. Each platform needs a different extraction technique, which is why three approaches coexist inside the same Python codebase.

main() - CLI entry point

Route / File	main.py
Language / Tech	Python - argparse
ML / AI Model	None - orchestrator; a job-title relevance checker (_matches_job_role) filters results with word-overlap logic

Parses --job-role, --location, --output, --limit, --cache-html and --save-html, runs the three scrapers in sequence, filters every result for role relevance, de-duplicates by URL and saves the final list using CSVStorage.

NaukriScraper.scrape()

Route / File	scrapers/naukri_scraper.py
Language / Tech	Python - requests + BeautifulSoup + lxml, pandas for data
ML / AI Model	None - HTML structure parsing + pagination rules

Builds the Naukri search URL from the parsed query, fetches the HTML with requests (rotating through pages for pagination), parses job cards with BeautifulSoup CSS selectors and normalises each result into the Job dataclass. HTML can be cached for offline re-runs via utils/html_cache.py.

RemoteOKScraper.scrape()

Route / File	scrapers/remotek_scraper.py
Language / Tech	Python - requests against the public JSON API
ML / AI Model	None - JSON mapping; a keyword-relevance heuristic keeps technical roles in the results

Fetches https://remotek.com/api directly, selects jobs up to the requested limit, maps the flat JSON fields (position, company, location, salary, url, date, description) onto the Job model and filters by role keywords when needed.

WellfoundScraper.scrape()

Route / File	scrapers/wellfound_scraper.py
Language / Tech	Python - Playwright (headless Chromium) + Firecrawl
ML / AI Model	None - browser automation + network-response capture; not an ML model, but it renders JavaScript so the site's client-side logic executes

Launches a headless browser, registers a response listener that captures every JSON payload containing 'jobs', opens the Wellfound search URL and waits for the network calls to finish. This API-interception technique survives modern JavaScript-rendered SPAs and returns structured JSON without fragile DOM parsing. Firecrawl-py is available as a fallback service.

CSVStorage.save_to_csv() and Job/JobValidator models

Route / File	storage/csv_storage.py, models/job_model.py
Language / Tech	Python - csv module + dataclasses
ML / AI Model	None - data validation rules

Standardises scraped records (cleaning text, verifying URL protocols, checking required fields and allowed source platforms) and exports them to a CSV with columns job_title, company_name, location, job_url, salary, description, posted_date and source.

9. Database Layer (SQLAlchemy Models)

All persistence is handled by Flask-SQLAlchemy ORM. In development the engine uses SQLite; in production DATABASE_URL switches to PostgreSQL with connection pooling (pool_pre_ping, pool_size 10, max_overflow 20). The schemas below are the 'memory' of every function described so far.

Model / Table	Key Columns	Functions It Backs
User (users)	email, username, password_hash, login_attempts, locked_until	signup, login, lockout, password reset
UserProfile (user_profiles)	full_name, phone, location, headline, linkedin/github/portfolio, resume_text, skills	create/auth/me, profile view
UserPreference	language, theme, weekly_digest, search_radius_km, preferred_locations	settings, job alert scoping
SavedJob	role_title, company, status, guide_data	dashboard saved guides, save/remove-job APIs
JobApplication	company, role, status, applied_date	application tracker stats
InterviewHistory	company, role, interview_type, status, date	interview tracker stats
UserSession	user_id, token, is_active	active-session invalidation on password reset
AuditLog	action, resource_type/ID, ip_address	full audit trail for every security event
PasswordResetToken	token, expires_at, is_used	forgot/reset password flow
ChatMessage	user_id, message, message_type, is_flagged	community chat stream + moderation
ChatUserStatus	has_accepted_policy, is_banned, ban_reason	chat policy gate and ban screen
Job (dataclass, not a table)	job_title, company, location, salary, source	scraper normalisation + validation

10. n8n AI Orchestration Workflows

The n8n/ folder contains five automation workflows defined in JSON (the JS-box language of n8n is JavaScript). Each workflow listens on a webhook, calls the Google AI node with gemini-pro, post-processes the output with JavaScript code nodes and writes the result to PostgreSQL. This duplicates and extends the in-app AI capabilities described in Section 3.

Workflow File	Trigger	AI Model	Pipeline
resume_analysis_workflow.json	Webhook POST resume-analysis	gemini-pro (temperature 0.3)	Webhook -> Google AI -> JS parse/JSON-extract -> PostgreSQL
interview_prep_workflow.json	Webhook POST interview-prep	gemini-pro	Webhook -> Google AI -> JS structure -> Respond
career_insights_workflow.json	Webhook POST career-insights	gemini-pro	Webhook -> Google AI -> JS structure -> Respond
job_recommendations_workflow.json	Webhook POST recommendations	gemini-pro	Webhook -> Google AI -> JS structure -> Respond
feedback_processing_workflow.json	Webhook POST feedback	gemini-pro	Webhook -> Google AI (sentiment/summary) -> JS structure

The JS Code nodes include a JSON-extraction routine (`const m = $json.message.content.match(/\[["']s/...)`) with a fallback structure that guarantees a valid response even when the LLM output is malformed - the same fail-soft design used in the Python layer. Making requests through n8n instead of Python lets operations staff rewire prompts and model versions without deploying code.

12. Hardware-Constrained Architecture Plan & Proposed Local AI Stack

Full design is in docs/ARCHITECTURE.md. The development machine is an Intel Core i7-6820HQ (4 cores / 8 threads) with 32 GB RAM and a Quadro M2000M 4 GB GPU. The plan forbids running an LLM for every operation; each concern gets a specialised lightweight component, and any LLM work goes through a swappable LocalLLMProvider abstraction instead of hard-coding Ollama.

12.1 Specialised lightweight components (languages + models per function)

#	Function	Implementation (Language + Local Model)	Runtime
1	Resume PDF/DOCX extraction	Python - pdfplumber, python-docx, PyPDF2 (local document parser)	CPU
2	Resume entity extraction	Python - local NER: spaCy small model or GLiNER (tiny/quantized); regex fallback for email/phone/LinkedIn/GitHub	CPU
3	Skill extraction & normalization	Python - NLP + skill taxonomy + rules/ML: SKILLS_DATABASE regex match + synonym alias table, optional TF-IDF	CPU
4	Resume <-> Job matching	Python - sentence-transformers all-MiniLM-L6-v2 (or ONNX-quantized MiniLM); cosine similarity + deterministic keyword scoring	CPU
5	ATS scoring	Python - existing deterministic explainable weighted engine	CPU
6	Skill-gap analysis	Python - rules + matching engine (LEARNING_RESOURCES + taxonomy normalization)	CPU
7	Market-demand analysis	Python - statistics computed from real Adzuna/JSearch job data (counts, counts median/percentiles); no generative AI	CPU
8	Recommendations	Python - rules + ML where useful; precomputed embedding similarity in batches batched per user profile	CPU
9	Interview question generation	Python - local small/quantized LLM (e.g. Qwen2.5-Instruct 1.5B/3B, Llama 3.2 1B/3B, Qwen2.5 1.5B/3B, Qwen2.5 1.5B/3B, Qwen2.5 1.5B/3B) via LocalLLMProvider	CPU
10	Interview evaluation	Python - rubric keyword/coverage scoring first; local LLM only for open-ended free-text and only if uncached input	CPU

12.2 Where LLM inference is allowed vs forbidden

Allowed: interview question generation (small quantized local LLM); interview evaluation for open-ended free-text only, after rubric scoring, only when not cached; optional occasional 'explain my result' generation.

Forbidden: page loads, job matching, skill comparisons, database operations, ATS scores, skill-gap computation, market-demand stats, recommendations and resume parsing never call an LLM.

12.3 LocalLLMProvider abstraction

Backend never imports a provider SDK directly. A Python ABC (chat, complete, optional embed, health, model_info, metrics) plus a get_provider() factory sit in a planned services/llm/ package. Supported backends: Ollama, llama.cpp llama-server, LM Studio, Jan and any OpenAI-compatible local endpoint - selected by LOCAL_LLM_PROVIDER and LOCAL_LLM_BASE_URL. Because the interface is HTTP-shaped, inference can later move to a separate server for many concurrent users by changing one config value.

12.4 Caching strategy (avoid repeated inference)

- In-memory layer:** cachetools.TTLCache for hot results in the same process (e.g. repeated resume texts in one session).

- **Database layer (planned SQLAlchemy models):** ResumeAnalysisCache (per resume_text_hash + target_role: ats_score, entities, skills, embeddings, provider, model, expiry), JobAnalysisCache (per job_url_hash: embeddings, requirements, matched roles, model_used, expiry) and EmbeddingCache (unique text_hash + model_name -> vector).
- **Precomputed batches:** nightly jobs pre-embed the job corpus and pre-score popular role/resume pairs so the web layer only reads prepared rows.
- Keys are sha256 content hashes of normalized text, so identical input never runs inference twice. TTL defaults: resume analysis 24 h, job analysis 7 d, embeddings until model version changes (a model column invalidates stale rows).

12.5 Batch & asynchronous processing

Resume NER, skill extraction and embedding runs process batches of documents (new jobs nightly, corpus re-embedding weekly) instead of running per request. Flask routes respond fast; long analysis is delegated to asyncio or a small worker queue and the client polls or is notified when ready.

Market-demand statistics are recomputed by scheduled batch jobs from the cached job corpus, never via fresh model calls inside a request. Celery/RQ are optional later, not required on the constrained machine.

12.6 Configurability (planned settings)

LOCAL_LLM_PROVIDER (ollama | llama_cpp | lm_studio | jan | openai_compatible),
 LOCAL_LLM_BASE_URL, LOCAL_LLM_MODEL, LOCAL_LLM_QUANT, LOCAL_LLM_MAX_TOKENS,
 LOCAL_LLM_TIMEOUT, LOCAL_EMBEDDING_MODEL, LOCAL_EMBEDDING_DEVICE,
 CACHE_RESUME_ANALYSIS_DB, CACHE_JOB_ANALYSIS_DB.

12.7 Benchmark-first runtime decision

No replacement runtime is chosen yet. The plan benchmarks Ollama, llama.cpp llama-server, LM Studio and Jan on the dev machine with small quantized GGUF models (Qwen2.5-Instruct 1.5B/3B, Llama 3.2 1B/3B, Phi-3-mini in Q4_K_M/Q8_0), measuring tokens/sec, first-token latency, peak RAM (< 8 GB), sustained CPU (< 60%), VRAM (< 3 GB) and temperature (< 85 C) over a 10-request test. Adoption thresholds: role question pack under 5 s on CPU and passing all resource limits; otherwise the curated INTERVIEW_QUESTIONS_DB runs 100% offline with no local LLM at all. Results recorded per run in docs/BENCHMARKS.md.

12.8 Future proofing / separate server

Migration order: (1) ship all dependency-free offline features; (2) add LocalLLMProvider + factory, benchmark, select runtime, flip config; (3) interview generation and evaluation use only the provider interface; (4) on bigger hardware or for many users, point LOCAL_LLM_BASE_URL at a dedicated inference server - no application code changes.

13. Appendix - Sibling Projects in the Workspace

The workspace folder also contains two additional projects. They are separate applications but live under the same directory tree - listed here for completeness.

job-platform (investment tracking platform)

Route / File	job-platform/app.py and job-platform/static/app/index.html
Language / Tech	Python Flask backend + JavaScript/HTML/CSS frontend + SQLite database (platform.db)
ML / AI Model	None - however it interacts with market-data APIs; SIP simulation uses compounding math (future-value formula) rather than ML forecasting

A full investment platform with watchlists, portfolios and holdings CRUD, an SIP (Systematic Investment Plan) simulator, price/volume alerts, admin asset management and audit logs. Its own app.py defines portfolio, watchlist, alert, admin, config and SPA-routing endpoints.

farsan-showcase (Next.js product showcase site)

Route / File	farsan-showcase/src/app/page.tsx, layout.tsx, globals.css
Language / Tech	TypeScript (SOLID JSX/React), Next.js, Tailwind CSS
ML / AI Model	None - static marketing/showcase site

A Next.js 14-style App Router site with clearly separated server and client components ("use client" directives), a landing page, feature sections and product galleries. It uses TypeScript interfaces for typed props and Tailwind for styling.

End of guide. Every function described above can be verified directly in the project source code. The design principle is consistent across the whole system: heavy, explainable logic in Python rule engines for instant responses, and transformer models (gemini-pro, bart-large-mnli) reserved for the tasks that genuinely need generative or zero-shot intelligence.